

JNX-OS Conversation Starter - Heiliger Code

CRITICAL: Lies dies ZUERST in jeder neuen Konversation!

Projekt-Kontext

Projekt: JNX-OS v2.0 - Enterprise SaaS Foundation

Status:  Production-Ready (Vercel Deployed)

Stack: Next.js 14 + Clerk Auth + Supabase PostgreSQL + Tailwind CSS v3.3.3

Location: `/home/ubuntu/jnx-os/nextjs_space/`

ABSOLUTE RULES (Non-Negotiable)

1. PROTECTED FILES - NICHT ÄNDERN ohne expliziten Grund:

Authentication (Clerk Integration):

-  `lib/auth/clerk-client.ts`
-  `lib/auth/clerk-server.ts`
-  `lib/auth/helpers.ts`
-  `lib/auth/rbac.ts`
-  `middleware.ts`

Database (Idempotente Operationen):

-  `lib/db/helpers.ts` # UPSERT functions
-  `lib/db/schema-v2.sql` # Production schema
-  `MIGRATION_SIMPLE.sql` # GDPR migration

Security & GDPR:

-  `lib/security/rate-limit.ts`
-  `lib/security/headers.ts`
-  `lib/privacy/*` # redaction, export, deletion
-  `lib/observability/logger.ts`

Webhook (Kritisch für Clerk  Supabase Sync):

-  `app/api/webhooks/clerk/route.ts`

2. FORBIDDEN ACTIONS (Wird Production kaputt machen):

Database:

- Nie `INSERT` ohne `UPSERT` verwenden (Race Conditions!)
- Nie `DELETE` direkt (nur soft-delete via `deleted_at`)
- Nie `clerk_user_id` oder `clerk_org_id` Spalten entfernen
- Nie Schema ändern ohne Migration-File

Authentication:

- Nie Webhook-Signature-Verification überspringen

- Nie `requireAuth()` auf protected Routes weglassen
- Nie Custom Session Management bauen

✗ Security:

- Nie Rate Limiting deaktivieren
- Nie PII ohne Redaction loggen
- Nie Security Headers entfernen

✗ Code Quality:

- Nie `any` Type verwenden (use proper types)
- Nie `@ts-ignore` ohne Kommentar
- Nie Error Handling entfernen

3. MUST USE Patterns:

✓ Database Operations:

```
// IMMER diese Functions verwenden:
import { upsertUser, upsertOrg, createUserWithOrg } from '@lib/db/helpers';

// NIE direktes INSERT:
await upsertUser({ clerkUserId, email, fullName }); // ✓ Gut
await supabase.from('users').insert({...}); // ✗ Schlecht (Race Condition)
```

✓ Authentication (Server-Side):

```
import { requireAuth, requireAdmin } from '@lib/auth/helpers';

const { user, jnxUser } = await requireAuth(); // ✓ Immer verwenden
```

✓ Logging mit PII Redaction:

```
import { Logger } from '@lib/observability/logger';

const logger = new Logger('api/your-route');
logger.info('User action', { userId }); // ✓ Auto-redacted
```

🎯 Kritische System-Architektur

Authentication Flow (Clerk → Supabase):

1. User signs up via Clerk
↓
2. Clerk sends webhook `/api/webhooks/clerk`
↓
3. Webhook handler uses `createUserWithOrg()` (transactional, idempotent)
↓
4. User synced to Supabase
↓
5. Dashboard loads with server-side fallback (`syncUserFromClerk`)

Key Point: Dashboard funktioniert AUCH wenn Webhook verzögert ist (server-side fallback in `app/app/page.tsx`).

Database Schema (Clerk-Aligned):

```
-- Users (Multi-Tenant)
users (
  clerk_user_id TEXT UNIQUE NOT NULL, -- Clerk sync key
  org_id UUID REFERENCES orgs(id),
  deleted_at TIMESTAMPTZ,             -- GDPR soft-delete
  ...
)

-- Organizations
orgs (
  clerk_org_id TEXT UNIQUE,
  ...
)
```

Key Point: `clerk_user_id` ist die Source of Truth für User-Sync. NIEMALS ändern/entfernen.



Vollständige Dokumentation (Bei Unsicherheit lesen)

Vor Code-Änderungen:

1. **BACKEND_PROTECTION_PROMPT.md** (900 Zeilen)
 - Pre-Development Checklist
 - 5 Sections (Auth, Database, Security, API, UI)
 - Forbidden Actions
 - Safe Patterns
2. **QUICK_REFERENCE.md** (300 Zeilen)
 - Code-Snippets (Copy-Paste ready)
 - Testing Commands
 - Troubleshooting
 - File Locations

Architektur & Setup:

- `README.md` - Projekt-Übersicht
- `docs/ARCHITECTURE.md` - System Design
- `docs/BACKEND_CONTRACT.md` - Non-Negotiable Rules
- `docs/CLERK_SETUP.md` - Clerk Konfiguration
- `docs/GDPR_COMPLIANCE.md` - Privacy Features

Quick Commands (Häufig gebraucht)

```
# Build Check (MUSS vor Deployment erfolgen)
cd /home/ubuntu/jnx-os/nextjs_space && yarn build

# Type Check
yarn tsc --noEmit

# Dev Server
yarn dev

# Prisma Client regenerieren
yarn prisma generate

# Database Browser
yarn prisma studio
```

Häufige Probleme (Quick Fix)

Problem: “Column does not exist” Error

Lösung: Run `MIGRATION_SIMPLE.sql` in Supabase SQL Editor

Problem: Webhook Failed in Clerk Logs

Lösung: Check `CLERK_WEBHOOK_SECRET` in `.env`, verify Supabase connection

Problem: Dashboard stuck on setup screen

Lösung:

1. Check Supabase connection
2. Verify user exists: `SELECT * FROM users WHERE clerk_user_id = 'user_xxx'`
3. Check webhook logs in Clerk Dashboard

Problem: Admin Panel shows “Unauthorized”

Lösung: Set user role in Clerk Dashboard → User → Public Metadata:

```
{ "role": "admin" }
```

Problem: Build fails with “Module not found”

Lösung: `cd nextjs_space && yarn install && yarn prisma generate`

Deployment Status

Current Deployment:

-  GitHub: JNXLabs/jnx-os (main branch)
-  Vercel: Auto-Deploy aktiv
-  Database: Supabase PostgreSQL (Schema v2)
-  Build: Passing (Tailwind CSS v3.3.3)

Environment Variables (Required):

```
CLERK_PUBLISHABLE_KEY=pk_test_xxx
CLERK_SECRET_KEY=sk_test_xxx
CLERK_WEBHOOK_SECRET=whsec_xxx
SUPABASE_URL=https://xxx.supabase.co
SUPABASE_ANON_KEY=eyJxxx
SUPABASE_SERVICE_ROLE_KEY=eyJxxx
```

! CRITICAL: Symlink-Problem & Vercel Deployments**Was ist das Problem?**

DeepAgent verwendet **Symlinks** für Effizienz (Zeiger auf zentrale Dateien). Diese funktionieren lokal, aber **NICHT auf Vercel**:

```
# Lokale Umgebung (funktioniert)
yarn.lock -> /opt/hostedapp/node/root/app/yarn.lock ✓

# Vercel Clone (funktioniert NICHT)
yarn.lock -> /opt/hostedapp/node/root/app/yarn.lock ✗
# Fehler: ENOENT: no such file or directory
```

Automatische Lösung (Installiert):**1. Git Pre-Push Hook**

- ✓ **Automatisch installiert** in `.git/hooks/pre-push`
- ✓ Prüft vor jedem `git push`, ob `yarn.lock` ein Symlink ist
- ✓ Konvertiert automatisch zu echter Datei
- ✓ Staged die Änderung automatisch

Was passiert beim `git push`:

```
$ git push origin main
🔍 Checking for symlinks before push...
✓ yarn.lock is already a real file
🚀 Ready to push to GitHub
```

2. Deployment Verification Script

- 📍 Location: `scripts/verify-deployment-ready.sh`
- ✓ Prüft Symlinks, Environment Variables, package.json
- ✓ Manuell vor wichtigen Deployments ausführen

Manuell ausführen:

```
cd /home/ubuntu/jnx-os
bash scripts/verify-deployment-ready.sh
```

Wenn der Fehler TROTZDEM auftritt:

Quick Fix (in 30 Sekunden):

```
# 1. Symlink durch echte Datei ersetzen
cd /home/ubuntu/jnx-os/nextjs_space
cp -L /opt/hostedapp/node/root/app/yarn.lock .

# 2. Zu Git hinzufügen
cd /home/ubuntu/jnx-os
git add nextjs_space/yarn.lock
git commit -m "fix: replace yarn.lock symlink with real file"
git push origin main

# 3. Vercel baut automatisch neu ✅
```

Warum passiert das?

DeepAgent-Optimierung:

Vorteile:

- ✅ Speicherplatz sparen (zentrale node_modules)
- ✅ Schnellere lokale Builds
- ✅ Konsistente Versionen

Nachteil:

- ❌ Symlinks funktionieren nicht auf Vercel

Lösung:

- Git Pre-Push Hook konvertiert automatisch
- Oder: Manuell mit obigem Quick Fix



TOP 7 Häufige Deployment-Probleme & Lösungen

Problem 1: Environment Variables fehlen auf Vercel ⚠️

Symptom:

```
Error: Invalid environment variables
TypeError: Cannot read property 'CLERK_SECRET_KEY' of undefined
```

Ursache:

- Lokale .env wird **NICHT** automatisch zu Vercel übertragen
- Environment Variables müssen manuell in Vercel konfiguriert werden

Lösung:

1. Gehe zu Vercel Dashboard → dein Projekt → Settings → Environment Variables
2. Füge ALLE Variablen aus .env hinzu:
 - NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY
 - CLERK_SECRET_KEY
 - CLERK_WEBHOOK_SECRET
 - NEXT_PUBLIC_SUPABASE_URL
 - NEXT_PUBLIC_SUPABASE_ANON_KEY
 - SUPABASE_SERVICE_ROLE_KEY
3. Wichtig: Wähle "Production", "Preview", und "Development" environments
4. Redeploy: "Deployments" → neuester Build → "..." → "Redeploy"

Prevention:

- ☒ Checklist vor jedem Deployment
- ☒ `.env.local.example` als Template pflegen

Problem 2: Prisma Client nicht generiert ⚠️**Symptom:**

```
Error: Cannot find module '@prisma/client'
Error: PrismaClient is unable to be run in the browser
```

Ursache:

- `prisma generate` nicht im Build-Process
- Prisma im falschen Dependency-Bereich (`devDependencies` statt `dependencies`)

Lösung (Quick Fix):

```
# Lokal testen
cd /home/ubuntu/jnx-os/nextjs_space
yarn prisma generate
yarn build

# Wenn erfolgreich, sicherstellen dass in package.json:
"dependencies": {
  "prisma": "6.7.0",
  "@prisma/client": "6.7.0"
}

# Committen und pushen
git add package.json
git commit -m "fix: ensure Prisma is in dependencies"
git push origin main
```

Vercel Build Script (Automatisch):

```
// package.json - sollte bereits vorhanden sein
"scripts": {
  "postinstall": "prisma generate",
  "build": "next build"
}
```

Problem 3: Clerk Webhook URL nicht auf Production gesetzt ⚠️**Symptom:**

- User können sich anmelden (Clerk funktioniert)
- Aber: Dashboard zeigt "Setting up your account" endless
- Database hat **KEINE** User-Einträge

Ursache:

- Webhook zeigt noch auf `localhost` oder falsche URL
- Clerk kann Production-App nicht erreichen

Lösung:

1. Gehe zu Clerk Dashboard → Webhooks
2. Überprüfe Endpoint URL:
 - ✗ `http://localhost:3000/api/webhooks/clerk`
 - ✓ `https://deine-app.vercel.app/api/webhooks/clerk`
3. Events müssen aktiviert sein:
 - ✓ `user.created`
 - ✓ `user.updated`
 - ✓ `organization.created`
 - ✓ `organization.updated`
 - ✓ `organizationMembership.created`
4. Test Webhook:
 - Clerk Dashboard → Webhooks → "... " → "Test"
 - Expected: 200 OK Response
5. Überprüfe Vercel Logs:
 - Vercel Dashboard → Deployment → Functions → Webhook Calls

Critical Check:

```
-- In Supabase SQL Editor
SELECT COUNT(*) FROM users;
-- Expected: > 0 nach erstem Signup

SELECT * FROM users ORDER BY created_at DESC LIMIT 5;
-- Expected: Deine Test-User sichtbar
```

Problem 4: Database Connection Pool Exhausted ⚠**Symptom:**

```
Error: remaining connection slots are reserved
Error: too many connections for role "postgres"
```

Ursache:

- Supabase Free Tier: Max 60 Connections
- Jeder Vercel Serverless Function öffnet neue Connection
- Connections werden nicht geschlossen

Lösung (Immediate):

1. Supabase Dashboard → Database → Connection Pooling
2. Enable Connection Pooler
3. Update `.env`:


```
SUPABASE_URL=https://xxx.supabase.co (Transaction mode)
# Oder für Connection Pooling:
DATABASE_URL=postgresql://postgres:[PROJECT_REF]:[PASSWORD]@aws-0-[REGION].pool-er.supabase.com:5432/postgres
```
4. Redeploy auf Vercel

Lösung (Long-term):

```
// lib/db.ts - Prisma mit Connection Pooling
import { PrismaClient } from '@prisma/client'

const globalForPrisma = global as unknown as { prisma: PrismaClient }

export const prisma = globalForPrisma.prisma || new PrismaClient({
  log: ['error'],
  datasources: {
    db: {
      url: process.env.DATABASE_URL
    }
  }
})

if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma
```

Problem 5: Build Cache führt zu veralteten Builds **Symptom:**

- Änderungen lokal sichtbar
- Nach Push zu GitHub: Build erfolgreich
- Aber: Production zeigt alte Version

Ursache:

- Vercel cached Build-Artefakte
- Next.js cached Pages/API Routes
- Änderungen werden nicht übernommen

Lösung:

1. Vercel Dashboard → dein Projekt → Deployments
2. Neuester Build → "... " (drei Punkte)
3. "Redeploy" →  "Use existing Build Cache" DEAKTIVIEREN
4. "Redeploy"

```
# Alternative (für zukünftige Deployments):
git commit --allow-empty -m "chore: force rebuild"
git push origin main
```

Prevention:

```
// next.config.js - Cache-Bust bei kritischen Änderungen
/** @type {import('next').NextConfig} */
const nextConfig = {
  // Disable caching für API Routes (wenn nötig)
  experimental: {
    isrMemoryCacheSize: 0
  },

  // Generiere unique Build ID
  generateBuildId: async () => {
    return `build-${Date.now()}`
  }
}
```

Problem 6: TypeScript Errors nur auf Vercel ⚠

Symptom:

```
# Lokal:
✓ Compiled successfully

# Vercel:
x Type error: Property 'xyz' does not exist on type 'ABC'
```

Ursache:

- Lokale tsconfig.json hat "strict": false
- Vercel verwendet strikte TypeScript-Einstellungen
- Oder: Type-Definitionen fehlen

Lösung:

```
# 1. Reproduziere lokal
cd /home/ubuntu/jnx-os/nextjs_space
yarn build
# Wenn erfolgreich → Vercel-spezifisches Problem

# 2. Überprüfe tsconfig.json
{
  "compilerOptions": {
    "strict": true, // Sollte aktiviert sein
    "skipLibCheck": false // Sollte deaktiviert sein für Checks
  }
}

# 3. Fix Type Errors
# Beispiel: User type inconsistency
interface PlainUser {
  id: string;
  email: string | null;
  firstName: string | null;
  lastName: string | null;
}

# 4. Test lokal
yarn tsc --noEmit # Type-Check ohne Build
```

Quick Workaround (NOT RECOMMENDED):

```
// next.config.js - NUR für Notfälle
module.exports = {
  typescript: {
    ignoreBuildErrors: true // ⚠ Nur temporär!
  }
}
```

Problem 7: OAuth Redirects funktionieren nicht ⚠**Symptom:**

- Google/GitHub Login öffnet sich
- Nach Success: Redirect zu `localhost` oder 404

Ursache:

- Clerk Redirect URLs nicht für Production konfiguriert
- `NEXTAUTH_URL` fehlt oder falsch

Lösung:

1. Clerk Dashboard → Paths
2. Update Redirect URLs:
 - Sign-in: `/login`
 - Sign-up: `/signup`
 - After sign-in: `/app`
 - After sign-up: `/app`
3. Clerk Dashboard → SSO Connections → Google
 - Authorized redirect URIs:
 - ✓ `https://deine-app.vercel.app/api/auth/callback/google`
 - ✓ `https://deine-app.vercel.app/*`
4. Vercel Environment Variables:
 - `NEXTAUTH_URL=https://deine-app.vercel.app`
 - `NEXTAUTH_SECRET=<generiere mit: openssl rand -base64 32>`
5. Redeploy

 Pre-Deployment Checklist

Vor jedem Production-Push:

✓

Environment Variables **in** Vercel konfiguriert

✓

Clerk Webhooks auf Production URL gesetzt

✓

Supabase Connection Pooling aktiviert

✓

Prisma **in** dependencies (nicht devDependencies)

✓

yarn.lock ist echte Datei (kein Symlink)

✓

Lokaler Build erfolgreich (yarn build)

✓

TypeScript Check erfolgreich (yarn tsc --noEmit)

✓

Database Schema up-to-date (MIGRATION_SIMPLE.sql)

✓

Clerk Redirect URLs aktualisiert

✓

.env.local.example auf dem neuesten Stand

Quick Verification Script:

```
cd /home/ubuntu/jnx-os
bash scripts/verify-deployment-ready.sh
```



Emergency Rollback Procedure

Wenn Production komplett broken ist:

1. Vercel Dashboard → Deployments

2. Finde letzte funktionierende Version (grüner Haken)

3. Klick auf "... " → "Promote to Production"

4. Bestätige Rollback

Fix lokal

git log --oneline # Finde letzte funktionierende Version

git revert <commit-hash> # Oder: git reset --hard <commit-hash>

git push origin main

Nach Fix

git push origin main # Neuer Deployment



Production Quality Metrics (Erreicht)

Metrik	Before	After
Webhook Success Rate	~70%	100%
Dashboard Load Time	8-15s	<3s
500 Errors	Häufig	0
Race Conditions	Ja	Nein
Infinite Loops	Ja	Nein

Workflow für neue Features

1. Planung (5-10 min):

1. Lies BACKEND_PROTECTION_PROMPT.md
2. Beantworte Pre-Development Checklist:
 - Auth betreffend?
 - **Database** betreffend?
 - Security betreffend?
 - API betreffend?
 - UI betreffend?
3. Lies relevante Sections

2. Entwicklung (30-60 min):

1. Öffne QUICK_REFERENCE.md
2. Kopiere relevante Code-Templates
3. Entwickle Feature
4. Teste lokal (yarn build)

3. Testing (10-15 min):

1. yarn build (MUSS erfolgen)
2. yarn tsc --noEmit (keine Errors)
3. Test Auth-Flow (**Login/Signup**)
4. Test Feature funktioniert
5. Check Console (keine Errors)

4. Deployment:

1. Git commit + push
2. Vercel auto-deploy
3. Monitor deployment **logs**
4. **Verify** production works

Was du JETZT machen kannst

Stabile Features (Touch nichts an):

- ☒ Clerk Authentication + Google SSO
- ☒ Role-Based Access Control (Admin/Member)
- ☒ Multi-Tenant Database (Orgs + Users)
- ☒ GDPR Compliance (Soft-Delete, Export, Deletion)
- ☒ Security (Rate Limiting, Headers, PII Redaction)
- ☒ Webhook Sync (100% idempotent)
- ☒ Dashboard mit Server-Side Fallback

Neue Features hinzufügen (Safe):

- ☒ Neue UI Pages (verwende JNX Dark components)
- ☒ Neue API Routes (verwende Template aus Quick Reference)

- ☒ Neue Database Tables (mit Migration-File)
- ☒ Business Logic (erweitere, modifiziere nicht Core)

Checklist: Vor JEDER Code-Änderung

```
[ ] Habe ich BACKEND_PROTECTION_PROMPT.md gelesen?
[ ] Betrifft meine Änderung Protected Files?
[ ] Verwende ich die richtigen Patterns (UPSERT, requireAuth)?
[ ] Habe ich Migration-File erstellt (wenn Schema-Änderung)?
[ ] Habe ich lokal getestet (yarn build)?
[ ] Keine TypeScript Errors?
[ ] Keine Console Errors im Browser?
[ ] Auth-Flow funktioniert noch?
```

Wenn alle ☒ → Deploy safe!

Emergency Contacts

Bei kritischen Problemen:

1. Check `BACKEND_PROTECTION_PROMPT.md` Section für dein Problem
2. Check `QUICK_REFERENCE.md` "Common Issues"
3. Check Vercel Deployment Logs
4. Check Clerk Webhook Logs
5. Check Supabase Logs

Letzte Resort: Restore zu letztem Checkpoint via Abacus AI

TL;DR (Zu Lang; Nicht Gelesen)

3 Goldene Regeln:

1.  **Schütze was funktioniert**
 - Keine Protected Files ändern
 - Immer idempotente Operationen (UPSERT)
 - Nie Security Features deaktivieren
2.  **Folge den Patterns**
 - Verwende Templates aus Quick Reference
 - Kopiere bewährte Code-Strukturen
 - Erweitere, modifiziere nicht Core
3.  **Teste vor Deployment**
 - Lokaler Build muss erfolgen
 - Auth-Flows testen
 - Keine TypeScript-Fehler

Bei Unsicherheit: Lies vollständige Doku in `BACKEND_PROTECTION_PROMPT.md`

Version: 2.0

Last Updated: 2024-12-27

Status:  Production Ready

Deployment: Vercel (Auto-Deploy from GitHub)

 **Du bist jetzt geschützt! Viel Erfolg beim Entwickeln!**